

線形代数 転置原理 imos 法

転置原理入門

1. 概要

転置原理は、ある問題の解法から別の問題（転置問題）の解法を機械的に導出するという、意外性のある面白い考え方です。

特に**多項式の多点評価**のアルゴリズムの定数倍高速化が転置原理を通して発見されたことが有名です。他には**形式的べき級数の合成**のアルゴリズムも、転置原理によって見通し良く導出できることが知られています。

本講座では転置原理についての入門的な解説を行います。また転置原理を競技プログラミング初心者レベルの易しい具体例に適用し、アルゴリズムが機械的に導出される様子を確認します。

転置原理は直感的な理解をしづらいこと、比較的高度なアルゴリズムの導出だけが有名であることもあり、本講座執筆時点で、熱心に学習しているユーザーは上級者にもあまり多くありません。本講座では易しい具体例を丁寧に扱うので、苦手意識のある方もぜひ挑戦してください。

2. 前提知識

累積和による区間和の計算を仮定します。行列の計算について知っていることが望ましいですが、知らない場合にはスキップ可能です。

3. 転置問題

次の 2 つの問題を考えます。数は整数である必要はありませんが、簡単のため整数としておきます。

【問題 1 (A)】

N, M を整数とします. NM 個の整数 $A_{i,j}$ ($1 \leq i \leq N, 1 \leq j \leq M$) が与えられます. また M 個の整数 x_j ($1 \leq j \leq M$) が与えられます. 次の式によって定まる N 個の整数 y_i ($1 \leq i \leq N$) を求めてください.

$$y_i = \sum_{j=1}^M A_{i,j} x_j.$$

【問題 1 (B)】

N, M を整数とします. NM 個の整数 $A_{i,j}$ ($1 \leq i \leq N, 1 \leq j \leq M$) が与えられます. また N 個の整数 Y_i ($1 \leq i \leq N$) が与えられます. 次の式によって定まる M 個の整数 X_j ($1 \leq j \leq M$) を求めてください.

$$X_j = \sum_{i=1}^N A_{i,j} Y_i.$$

このとき、**【問題 1(B)】** を **【問題 1(A)】** の**転置問題** (transposed problem) といいます. このとき **【問題 1(A)】** は **【問題 1(B)】** の転置問題です.

これらの関係を 2 つの方法で言い換えておきます. 行列の知識がない方は、言い換え 1 はスキップしてください.

3.1. 言い換え 1

x を $M \times 1$ 行列, y を $N \times 1$ 行列, A を $N \times M$ 行列とすると, **【問題 1(A)】** は $y = Ax$ を求める問題です. X を $M \times 1$ 行列, Y を $N \times 1$ 行列とすると, **【問題 1(B)】** は $X = A^T Y$ を求める問題です (A^T は A の転置行列を表すこととします).

つまり, **【問題 1(A)】** は行列 A を列ベクトルにかけると問題, **【問題 1(B)】** はその転置行列を列ベクトルにかけると問題です.

3.2. 言い換え 2

【問題 1(A)】 と 【問題 1(B)】 の x, y, X, Y は次を満たします：

$$\sum_{j=1}^M x_j X_j = \sum_{i=1}^N y_i Y_i.$$

よって 【問題 1(B)】 は、【問題 1(A)】 のような関係を持つような x, y に対して

$\sum_{j=1}^M x_j X_j = \sum_{i=1}^N y_i Y_i$ となる X_j を求める問題（ただし x, y は与えられず、 Y のみが与えられる）と言い換えられます。

X_j, Y_i は x_j, y_i の**双対変数**ということがあります。線形代数学では、特に行列の線形写像としての意味を重視する場合に、転置行列は双対写像の双対基底による表示と考えられています。その考え方からは言い換え 2 は言い換え 1 を書き直したものに過ぎません。

3.3. 例

x_1, x_2 を入力として、

$$y_1 = 2x_1 + 3x_2, \quad y_2 = 4x_1 + 5x_2, \quad y_3 = 6x_1 + 7x_2$$

を出力する問題を考えると、この問題の転置は Y_1, Y_2, Y_3 を入力として

$$X_1 = 2Y_1 + 4Y_2 + 6Y_3, \quad X_2 = 3Y_1 + 5Y_2 + 7Y_3$$

を出力する問題となります。このことを「言い換え 2」に基づいて導出してみましょう。

「言い換え 2」によれば、転置問題は Y_1, Y_2, Y_3 を入力として

$$x_1 X_1 + x_2 X_2 = (2x_1 + 3x_2) Y_1 + (4x_1 + 5x_2) Y_2 + (6x_1 + 7x_2) Y_3$$

が（任意の x_1, x_2 で）成り立つような X_1, X_2 を出力する問題です。右辺を x_1, x_2 について整理することで上の通りの X_1, X_2 が得られます。

注意

「 x を入力として y を出力せよ」という形式の問題すべてで転置問題が考えられるわけではありません。例えば $y_1 = x_1^2$ や $y_1 = \min(x_1, x_2)$ などの関係では転置問題は考えられません。

4. 転置原理

4.1. 転置原理

転置原理 (Transposition Principle, Tellegen's Principle) とは, **【問題 1 (A)】** を解くある種のアлゴリズムは, 機械的に **【問題 1 (B)】** を解くアルゴリズムに変換できるというものです. このアルゴリズムは, ネットワークの形式で解説されることが一般的だと思いますが, ここでは少し簡略化した方法で解説をします.

【定理 2】(転置原理)

【問題 1 (A)】 が次のアルゴリズムで解けるとする.

1. 変数 $z_1, \dots, z_k$ を 0 で初期化する.
2. $z_{s_1}, \dots, z_{s_M}$ に $x_1, \dots, x_M$ を代入する.
3. $q = 1, 2, \dots, Q$ に対して次の操作を行う:
 z_{a_q} に $c_q \times z_{b_q}$ を加える.
4. $y_1, \dots, y_N$ に $z_{t_1}, \dots, z_{t_N}$ を代入する.

このとき **【問題 1 (B)】** は次のアルゴリズムで解ける.

1. 変数 $Z_1, \dots, Z_k$ を 0 で初期化する.
2. $Z_{t_1}, \dots, Z_{t_N}$ に $Y_1, \dots, Y_N$ を代入する.
3. $q = Q, Q - 1, \dots, 1$ に対して次の操作を行う:
 Z_{b_q} に $c_q \times Z_{a_q}$ を加える.
4. $X_1, \dots, X_M$ に $Z_{s_1}, \dots, Z_{s_M}$ を代入する.

2 通りの方法で証明をします. 行列の知識がない方は, 証明 1 はスキップしてください.

4.2. 証明 1

「言い換え 1」を用います. 定理における **【問題 1 (A)】** を解くアルゴリズムは, すべて行列を使って表すことができます.

- z への x の代入: (s_j, j) 成分が 1, それ以外が 0 であるような行列 S を用いると, $z = Sx$ と書ける.

- q ステップ目の反復：対角成分は 1, (a_q, b_q) 成分が c_q , それ以外の成分が 0 であるような行列 A_q を用いると, q ステップ目の反復は z を $A_q z$ に置き換えることとして書ける.
- y への z の代入： (i, t_i) 成分が 1, それ以外が 0 であるような行列 T を用いると, $y = Tz$ と書ける.

このことから, **【問題 1(A)】** のアルゴリズムは

$$y = T \cdot A_Q A_{Q-1} \cdots A_2 A_1 \cdot S \cdot x$$

と書くことができます. **【問題 1(A)】** とは $y = Ax$ を求める問題でした. よってこのアルゴリズムの出力が任意の x に対して正しいとき,

$$A = T \cdot A_Q A_{Q-1} \cdots A_2 A_1 \cdot S$$

という行列積の関係があることになります. 転置行列の性質 $(AB)^T = B^T A^T$ から,

$$A^T = S^T \cdot A_1^T A_2^T \cdots A_{Q-1}^T A_Q^T \cdot T^T$$

が成り立ちます. したがって $X = A^T Y$ であるとき

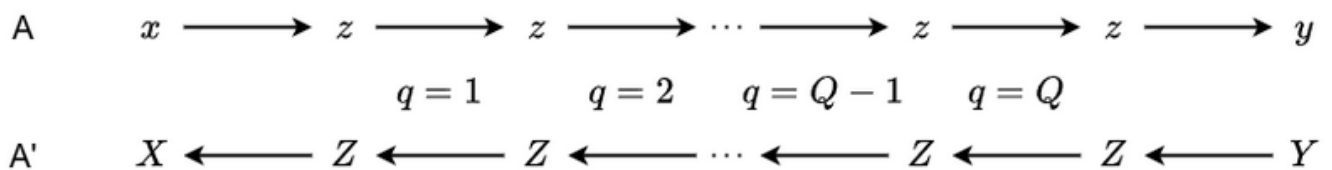
$$X = S^T \cdot A_1^T A_2^T \cdots A_{Q-1}^T A_Q^T \cdot T^T \cdot Y$$

が成り立ちます. これは **【問題 1(B)】** が Y に対して $T^T, A_Q^T, A_{Q-1}^T, \dots, A_2^T, A_1^T, S^T$ をこの順に左からかけていくことで解けることを表しています. このことを実際に書き下したのが定理における **【問題 1(B)】** のアルゴリズムです.

4.3. 証明 2

「言い換え 2」を用います.

定理における **【問題 1(A)】** と **【問題 1(B)】** のアルゴリズムを比べると, **【問題 1(B)】** のアルゴリズムは **【問題 1(A)】** のアルゴリズムの各段階を遡る方向に計算を進めていることが分かります.



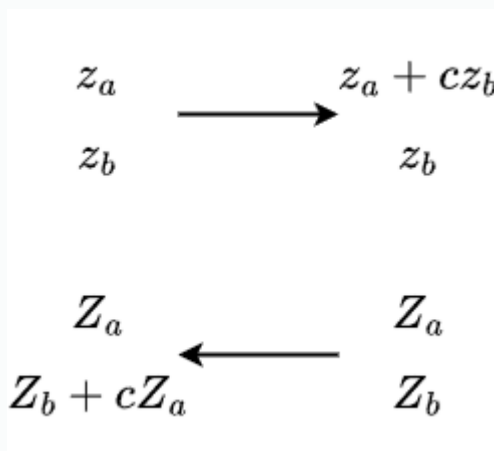
言い換え 2 より, 定理の **【問題 1(B)】** のアルゴリズムの出力を X としたとき, $\sum_{j=1}^M x_j X_j = \sum_{i=1}^N y_i Y_i$ が成り立つことを示せばよいです. これを次のように示します.

- 最初の x を z , Z を X に代入する部分で $\sum_i x_j X_j = \sum_k z_k Z_k$.
- $q = 1, 2, \dots, Q - 1, Q$ 回目の操作前後で, $\sum_k z_k Z_k$ は不変である.
- 最後の z を y , Y を Z に代入する部分で $\sum_k z_k Z_k = \sum_i y_i Y_i$.

証明のそれぞれの部分は非常に易しい計算です. 特に, 2 番目の部分については

$$(z_a + cz_b)Z_a + z_b Z_b = z_a Z_a + z_b (Z_b + cZ_a)$$

という式変形から確認できます.



5. 具体例：区間和クエリの転置

5.1. 区間和クエリの転置

次の問題を考えてみましょう.

【問題 3 (A)】

長さ N の整数列 $x = (x_0, x_1, \dots, x_{N-1})$ が与えられます.

Q 個のクエリに答えてください.

i 番目のクエリでは整数 L_i, R_i (ただし $0 \leq L_i \leq R_i \leq N - 1$) が与えられるので,

$y_i = \sum_{j=L_i}^{R_i} x_j$ を求めてください.

この問題の転置問題はどのようなになるでしょうか？考えてみてください。

転置問題は次のようになります。

【問題 3 (B)】

Q 個のクエリが与えられます。

i 番目のクエリでは整数 L_i, R_i, Y_i が与えられます (ただし $0 \leq L_i \leq R_i \leq N - 1$)。

次のようにして定まる N 個の整数 $X_0, X_1, \dots, X_{N-1}$ を求めてください: X_j は, $L_i \leq j \leq R_i$ を満たす i にわたる Y_i の総和である。

つまり, 【問題 3 (A)】 は区間和クエリに答える問題で, 【問題 3 (B)】 は区間加算クエリに答える問題です。

【問題 3 (A)】 は, **累積和** (prefix sum) を用いた解法が有名です。この解法に転置原理を用いて, 機械的に 【問題 3 (B)】 の解法を導出してみましょう。

5.2. 【問題 3 (A)】 の解法

【問題 3 (A)】 は次のように解くことができます。

- 変数 $s_0, s_1, \dots, s_N, y_1, y_2, \dots, y_Q$ を 0 で初期化する。
- s_{j+1} に x_j を代入する。
- $j = 0, 1, \dots, N - 1$ に対して次を行う:
 - s_{j+1} に s_j を加える。
- $i = 1, 2, \dots, Q$ に対して次を行う:
 - y_i に s_{R_i+1} を加える。
 - y_i に $-s_{L_i}$ を加える。
- $y_1, \dots, y_Q$ を出力する。

この解法は累積和の基本的な応用なので, これ以上解説はしません。

5.3. 【問題 3 (B)】 の解法

さて、この解法に転置原理を適用すると次のような 【問題 3 (B)】 の解法になります。

- 変数 $S_0, S_1, \dots, S_N, Y_1, Y_2, \dots, Y_Q$ を 0 で初期化する.
- $Y_1, \dots, Y_Q$ に問題文で与えられる入力を代入する.
- $i = 1, 2, \dots, Q$ に対して次を行う：
 - S_{L_i} に $-Y_i$ を加える.
 - S_{R_i+1} に Y_i を加える.
- $j = N - 1, \dots, 1, 0$ に対して次を行う：
 - S_j に S_{j+1} を加える.
- $X_0, X_1, \dots, X_{N-1}$ に $S_1, S_2, \dots, S_N$ を代入し、これを出力する.

【問題 3 (A)】 の解法を機械的に 【問題 3 (B)】 の解法に変換することができました。

5.4. 補足

おそらく 【問題 3 (B)】 の解法として最も有名な解法は、次の解法です。日本では imos 法として知られる方法です。

- 変数 $S_0, S_1, \dots, S_N, Y_1, Y_2, \dots, Y_Q$ を 0 で初期化する.
- $Y_1, \dots, Y_Q$ に問題文で与えられる入力を代入する.
- $i = 1, 2, \dots, Q$ に対して次を行う：
 - S_{L_i} に Y_i を加える.
 - S_{R_i+1} に $-Y_i$ を加える.
- $j = 0, 1, \dots, N - 1$ に対して次を行う：
 - S_{j+1} に S_j を加える.
- $X_0, X_1, \dots, X_{N-1}$ に $S_0, S_1, \dots, S_{N-1}$ を代入し、これを出力する.

5.3. 節で得られた解法は、この解法とループ順などに少し違いがありますが、どちらの解法も正しく動作します。

同じように、この 【問題 3 (B)】 の解法を転置することで、【問題 3 (A)】 についてループ順などを少し変えた別解法を得ることも可能です。試してみてください。

6. 練習問題

次の問題の転置問題がどのようなものであるかを考えてください。さらに、問題の解法を知っている場合には、その解法から転置問題の解法を導出してみてください。

【問題 4 (A)】(部分集合関係に関するゼータ変換)

整数列 $x = (x_0, x_1, \dots, x_{2^N-1})$ が与えられます。次のようにして定まる整数列 $y = (y_0, y_1, \dots, y_{2^N-1})$ を出力してください。

$$y_i = \sum_{j \subseteq i} x_j.$$

【問題 5 (A)】(約数関係に関するゼータ変換)

整数列 $x = (x_1, x_2, \dots, x_N)$ が与えられます。次のようにして定まる整数列 $y = (y_1, y_2, \dots, y_N)$ を出力してください。

$$y_i = \sum_{j|i} x_j.$$

【問題 6 (A)】(変更のある列に対する区間和クエリ)

整数列 $x = (x_0, x_1, \dots, x_{N-1})$ が与えられます。 Q 個のクエリを処理してください。クエリは次の 2 種があります。

- 1 $i \ v$: x_i に v を加える。
- 2 $L \ R$: $x_L + \dots + x_R$ を出力する。

7. 練習問題の簡易解説

6 節の練習問題について、簡易的に解説します。

【問題 4 (A)】

転置問題は、上位集合関係に関するゼータ変換です。

【問題 4 (B)】

整数列 $Y = (Y_0, Y_1, \dots, Y_{2^N-1})$ が与えられます。次のようにして定まる整数列 $X = (X_0, X_1, \dots, X_{2^N-1})$ を出力してください。

$$X_i = \sum_{j \supseteq i} Y_j.$$

部分集合関係に対する高速ゼータ変換のアルゴリズムから、上位集合関係に関する高速ゼータ変換のアルゴリズムを機械的に導出可能です。

【問題 5 (A)】

転置問題は、倍数関係に関するゼータ変換です。

【問題 5 (B)】

整数列 $Y = (Y_1, Y_2, \dots, Y_N)$ が与えられます。次のようにして定まる整数列 $X = (X_1, X_2, \dots, X_N)$ を出力してください。

$$X_i = \sum_{i|j} Y_j.$$

約数関係に対する高速ゼータ変換のアルゴリズムから、倍数関係に関する高速ゼータ変換のアルゴリズムを機械的に導出可能です。

【問題 6 (A)】

q 回目のクエリがタイプ 1 のクエリであるとき、 v を v_q と書くことにします。 q 回目のクエリがタイプ 2 のクエリであるとき、出力するべき答えを y_q と書くことにします。

y_q は x_i や $v_{q'}$ ($q' < q$) のうちいくつかの和なので、転置問題は、 X_i や $V_{q'}$ ($q' < q$) のうちいくつかに Y_q を足すという形になります。

クエリを逆順に考えると、転置問題は次の 2 種のクエリを処理する問題ということができます。

【問題 6 (B)】

整数列 $X = (X_0, X_1, \dots, X_{N-1})$ があり、はじめはすべての項が 0 です。 Q 個のクエリを処理してください。クエリは次の 2 種があります。

- 1 $i \ v$: X_i を出力する。
- 2 $L \ R \ Y$: $X_L, \dots, X_R$ に Y を加算する。

さらに最後に、すべてのクエリが終了した時点での $X_0, \dots, X_{N-1}$ を出力してください。

例えば 【問題 6 (A)】 に対するセグメント木・Fenwick 木による解法を転置すれば、【問題 6 (B)】 に対する双対セグメント木・双対 Fenwick 木による解法が得られます。

8. 関連問題

1. https://atcoder.jp/contests/arc185/tasks/arc185_e
2. https://atcoder.jp/contests/xmascon22/tasks/xmascon22_f
3. <https://qoj.ac/contest/1195/problem/6184>
4. https://atcoder.jp/contests/joi2026final/tasks/joi2026final_e
5. <https://codeforces.com/gym/102978/problem/D>

転置原理を用いて問題を考察するという考え方はしばしば有用であるものの、結果的に転置原理を経由しない解法に翻訳可能であることが多いため、解説で転置原理について触れられている競技プログラミングの問題はあまり多くはありません。公式解説やユーザー解説で転置原理に触れている問題についていくつか紹介します。ただしこれらの問題は 1 を除き、難易度はかなり高めです。

4, 5 は転置原理を意識して作問された問題です。特に 5 は、明確に転置原理を意識して作問された問題としては、競技プログラミングコミュニティで初めてのものだとする説もあるようです。

9. まとめ

本講座では、転置原理について解説し、実際に転置原理を区間和クエリに適用することで、区間加算クエリのアルゴリズムが機械的に導出できることを確認しました。

このように問題、解法を変換するという考え方はほとんど見られない面白いものだと思います。理解に時間がかかるかもしれませんが、ぜひこの機会に学んでみてください。

AtCoder Algorithm Lectures では、多項式の多点評価や形式的べき級数の合成などの転置原理を用いた導出も今後解説する予定です。

10. 参考文献

1. Nachia. 転置原理まとめ. <https://www.mathenachia.blog/tellegens/>. (閲覧日: 2026-04-01).
2. maspy. FPS 合成・逆関数の解説 (2) 転置原理による合成アルゴリズムの導出. [https://maspy.py.com/fps-合成・逆関数の解説 \(2\) 転置原理による合成アルゴリズムの導出](https://maspy.py.com/fps-合成・逆関数の解説(2)転置原理による合成アルゴリズムの導出). (閲覧日: 2026-04-01).
3. Alin Bostan, Grégoire Lecerf, Éric Schost. Tellegen's Principle into Practice. Proceedings of the 2003 International Symposium on Symbolic and Algebraic Computation (ISSAC '03). pp. 37–44. 2003. DOI: 10.1145/860854.860870. <https://doi.org/10.1145/860854.860870>.
4. Ryuhei Mori. $X^N \bmod P(X)$ の計算. <https://qiita.com/ryuhe1/items/c18ddbb834eed724a42b>. (閲覧日: 2026-04-01).

トップページ : [AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など : [Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。 リアルタイムのオンラインコンテストで競い合うことや、 5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

各サービス

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

[利用規約](#)